



Generación y Optimización de Código (80 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



➤ Creación de funciones y algoritmos con apoyo de IA.

- I. Generación de funciones simples*
- II. Generación de lógica compleja*
- III. Generación de algoritmos avanzados*
- IV. Estrategia de 3 niveles para crear funciones*
- V. Buenas prácticas al generar algoritmos con IA*
- VI. Errores comunes al generar algoritmos con IA*



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

La IA puede generar desde funciones simples hasta algoritmos avanzados, pero como programadores debemos guiarla con precisión, validar la complejidad y asegurar que el código es correcto, seguro y eficiente.

Es fácil decirle a la IA lo que deseamos, pero no pidas "un algoritmo de ordenamiento"; pide una función que procese un array de objetos `Task`, valide fechas y maneje nulos.

3.1 Generación de funciones simples

La IA es especialmente eficaz generando funciones:

- ☐ Validaciones
- ☐ Conversores
- ☐ Parsers básicos
- ☐ Operaciones matemáticas
- ☐ Manipulación de strings
- ☐ Utilidades comunes

■ 3.1 Generación de funciones simples

Ejemplos típicos:

- ☐ Validar emails
- ☐ Convertir temperaturas
- ☐ Capitalizar palabras
- ☐ Calcular promedios
- ☐ Filtrar listas

Prompt recomendado:

"Crea una función en Python que valide emails usando regex, con docstring y type hints."

◆ 1. Validar Emails

```
import re
from typing import Optional

def validar_email(email: str) -> bool:
    """
    Valida si un email tiene formato correcto usando regex.

    Args:
        email (str): Dirección de email a validar

    Returns:
        bool: True si el email es válido, False en caso contrario

    Ejemplos:
        >>> validar_email("usuario@ejemplo.com")
        True
        >>> validar_email("usuario@com")
        False
        >>> validar_email("usuario.ejemplo.com")
        False
    """
    patron = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(patron, email))
```



"Crea una función en [LENGUAJE] que haga [TAREA], con:

1. Docstring explicando uso
2. Type hints para parámetros
3. Ejemplos en la documentación
4. Manejo de casos límite"

- ✓ Código reutilizable
- ✓ Fácil de entender
- ✓ Bien documentado
- ✓ Fácil de testear
- ✓ Tipado para evitar errores

3.2 Generación de lógica compleja

Cuando la función requiere más pasos, condiciones o estructuras, la IA puede:

- ☐ Procesar arrays y objetos
- ☐ Manejar casos Edge
- ☐ Implementar lógica de negocio
- ☐ Crear parsers
- ☐ Gestionar permisos y roles

■ 3.2 Generación de lógica compleja

Ejemplos:

- ☐ Procesar transacciones
- ☐ Calcular balances
- ☐ Analizar logs
- ☐ Crear un parser matemático

Prompt recomendado:

"Crea una función que procese un array de transacciones y calcule el balance final, manejando múltiples monedas con tasas de conversión."

◆ Código Minimalista



```
def calcular_balance(transacciones, tasas):  
    total = 0  
    for monto, moneda, tipo in transacciones:  
        # Convertir a USD  
        if moneda != "USD":  
            monto = monto * tasas[moneda]  
  
        # Sumar ingresos, restar egresos  
        total = total + monto if tipo == "ingreso" else total - monto  
  
    return round(total, 2)
```

```
# Datos de entrada  
transacciones = [  
    (100, "USD", "ingreso"),    # +$100  
    (50, "EUR", "egreso"),      # -€50 → -$53  
    (30, "USD", "egreso"),      # -$30  
    (80, "EUR", "ingreso")      # +€80 → +$84.8  
]  
  
# Tasas: 1 EUR = 1.06 USD  
tasas = {"EUR": 1.06}  
  
# Calcular  
resultado = calcular_balance(transacciones, tasas)  
print(f"Balance: ${resultado} USD") # Resultado: $101.8
```



◆ Versión con Clases (Un poco más completa)

- 1. Recorrer transacciones
- 2. Convertir a USD
- 3. Sumar/Restar según tipo
- 4. Retornar total

EJEMPLO 1:

Input: [(100, "USD", "ingreso"), (50, "EUR", "ingreso")]

Tasas: {"EUR": 1.06}

Output: \$47.0

EJEMPLO 2:

Input: [(200, "USD", "ingreso"), (100, "EUR", "ingreso")]

Tasas: {"EUR": 1.06}

Output: \$306.0

```
class Transaccion:
    def __init__(self, monto, moneda, tipo):
        self.monto = monto
        self.moneda = moneda # "USD", "EUR", "GBP"
        self.tipo = tipo     # "ingreso" o "egreso"

def calcular_balance(transacciones, tasas):
    total = 0

    for t in transacciones:
        # Convertir si es necesario
        if t.moneda != "USD":
            monto_usd = t.monto * tasas.get(t.moneda, 1)
        else:
            monto_usd = t.monto

        total += monto_usd if t.tipo == "ingreso" else total - monto_usd

    return round(total, 2)

# Uso
transacciones = [
    Transaccion(100, "USD", "ingreso"),
    Transaccion(50, "EUR", "egreso"),
    Transaccion(80, "EUR", "ingreso")
]
```

3.3 Generación de algoritmos avanzados

◆ Búsqueda

- ☐ BFS
- ☐ DFS
- ☐ Dijkstra
- ☐ A*

◆ Ordenamiento

- ☐ QuickSort
- ☐ MergeSort
- ☐ HeapSort
- ☐ RadixSort

◆ Estructuras de datos

- ☐ Árboles
- ☐ Grafos
- ☐ Heaps
- ☐ Tries

Prompt recomendado:

"Implementa búsqueda A* en Python para encontrar el camino más corto en un grafo, con heurística Manhattan."



```
from queue import PriorityQueue

def a_star(inicio, meta, grafo, heuristica):
    """A* para encontrar camino más corto"""
    frontera = PriorityQueue()
    frontera.put((0, inicio))

    costo_g = {inicio: 0}
    padres = {inicio: None}

    while not frontera.empty():
        actual = frontera.get()[1]

        if actual == meta:
            break

        for vecino, costo in grafo[actual].items():
            nuevo_costo = costo_g[actual] + costo
            if vecino not in costo_g or nuevo_costo < costo_g[vecino]:
                costo_g[vecino] = nuevo_costo
                prioridad = nuevo_costo + heuristica(vecino, meta)
                frontera.put((prioridad, vecino))
                padres[vecino] = actual

    return reconstruir_camino(padres, meta)
```



Ejemplo Práctico: Laberinto 3x3

```
# Grafo simple (laberinto 3x3)
grafo = {
    (0,0): {(0,1): 1, (1,0): 1},
    (0,1): {(0,0): 1, (0,2): 1, (1,1): 1},
    (0,2): {(0,1): 1, (1,2): 1},
    (1,0): {(0,0): 1, (1,1): 1, (2,0): 1},
    (1,1): {(0,1): 1, (1,0): 1, (1,2): 1, (2,1): 1},
    (1,2): {(0,2): 1, (1,1): 1, (2,2): 1},
    (2,0): {(1,0): 1, (2,1): 1},
    (2,1): {(1,1): 1, (2,0): 1, (2,2): 1},
    (2,2): {(1,2): 1, (2,1): 1}
}

def heuristica_manhattan(a, b):
    """Distancia Manhattan: |x1-x2| + |y1-y2|"""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def reconstruir_camino(padres, nodo):
    """Reconstruye el camino desde el inicio hasta el nodo"""
    camino = []
    while nodo is not None:
        camino.append(nodo)
        nodo = padres.get(nodo)
    return camino[::-1]

# Buscar camino de (0,0) a (2,2)
inicio = (0, 0)
meta = (2, 2)
camino = a_star(inicio, meta, grafo, heuristica_manhattan)

print(f"Camino de {inicio} a {meta}:")
print(" → ".join(str(p) for p in camino))
```

◆ Visualización del Laberinto

```
(0,0) — (0,1) — (0,2)
|         |         |
(1,0) — (1,1) — (1,2)
|         |         |
(2,0) — (2,1) — (2,2)
```

◆ Paso a Paso de A^*

INICIO: (0,0) META: (2,2)

PASO 1: Expandir (0,0)

- Vecinos: (0,1) y (1,0)
- $f(0,1) = g(1) + h(3) = 4$
- $f(1,0) = g(1) + h(3) = 4$

PASO 2: Expandir (0,1)

- Vecinos: (0,0), (0,2), (1,1)
- $f(0,2) = g(2) + h(2) = 4$
- $f(1,1) = g(2) + h(2) = 4$

... Continúa hasta encontrar (2,2)

◆ Resultado del Ejecución

Camino de (0,0) a (2,2):

(0,0) → (0,1) → (1,1) → (2,1) → (2,2)

o

(0,0) → (1,0) → (1,1) → (1,2) → (2,2)

Costo total: 4 (distancia Manhattan)



```
def a_star_simple(inicio, meta, grafo):  
    """Versión más detallada para explicar"""  
    frontera = [(0, inicio)] # (prioridad, nodo)  
    visitados = set()  
    padres = {}  
    costos = {inicio: 0}  
  
    while frontera:  
        frontera.sort() # Ordenar por prioridad  
        _, actual = frontera.pop(0)  
  
        if actual in visitados:  
            continue  
  
        visitados.add(actual)  
  
        # Si llegamos a la meta  
        if actual == meta:  
            break  
  
        # Explorar vecinos  
        for vecino, costo in grafo[actual].items():  
            nuevo_costo = costos[actual] + costo  
  
            if vecino not in costos or nuevo_costo < costos[vecino]:  
                costos[vecino] = nuevo_costo  
                h = abs(vecino[0]-meta[0]) + abs(vecino[1]-meta[1]) # Manhattan  
                f = nuevo_costo + h  
                frontera.append((f, vecino))  
                padres[vecino] = actual  
  
        # Reconstruir camino  
        camino = []  
        nodo = meta  
        while nodo != inicio:  
            camino.append(nodo)  
            nodo = padres[nodo]  
        camino.append(inicio)  
  
    return camino[::-1], costos[meta]
```

3.4 Estrategia de 3 niveles para crear funciones

Con esta modalidad de estrategia entendemos a pedir código de forma profesional.

- ◆ Nivel 1 — Funciones simples

Descripción breve.

Ejemplo: "Función que capitalice la primera letra de cada palabra."

- ◆ Nivel 2 — Lógica compleja

Inputs, outputs, casos edge.

Ejemplo: "Parser de expresiones matemáticas con soporte para paréntesis, operadores y variables."

- ◆ Nivel 3 — Sistemas completos

Arquitectura + patrones + restricciones.

Ejemplo: "Diseña un sistema de caché distribuido con estrategia LRU, replicación y failover automático en Go."

■ 3.5 Buenas prácticas al generar algoritmos con IA

- ☐ Pedir complejidad deseada
- ☐ Especificar restricciones
- ☐ Solicitar casos edge
- ☐ Pedir explicación paso a paso
- ☐ Validar con tests unitarios
- ☐ Comparar con implementaciones estándar



Gracias.



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



